GPU Accelerated SOEN Simulation

Miles Zheng

November 2024

1 Introduction

A spiking neural network (SNN) is a neural network made of biologically inspired neurons. A neuromorphic computer is being built, but a simulator is useful to test ideas and discover interesting behavior. Since neural networks are so large, running them can take days, so speed is a top priority. We can speed up code using GPUs that utilize parallel, Single Instruction Multiple Data (SIMD) architecture. The best way to harness GPUs is to write code with all of the data in large matrices and arrays, where calculations can be done in parallel, instead of sequentially.

A computer normally has a CPU that does all the computation, but a CPU works the fastest when it has to deal with smaller amounts of data, in the range of kilobytes. GPUs on the other hand, can work fast with gigabytes of data, so for simulating thousands of neurons, a GPU is the better alternative.

A Graphics Processing Unit (GPU) is a piece of computer hardware used for parallel processing. We used NVIDIA GPUs that use the CUDA programming language. GPUs are optimized for large amounts of data but relatively fewer instructions. This makes them ideal for dealing with problems like matrix multiplication, since there is a lot of independent multiplication and addition. In GPUs, data treated as blocks instead of single values.

Neurons have membrane potential that is accumulated as a neuron receives spikes. Once the membrane potential crosses a threshold the neuron fires a spike. We use the term signal to refer to current in the neuromorphic computer and membrane potential in a neuron. Signal is the thing being integrated. Signal is passed between dendrites, and then integrated within the neuron. In a soma in our simulation, once the signal is integrated and reaches a threshold of 0.7, a spike is fired to other neurons. The soma is similar to a dendrite, but it's behavior is different when signal reaches the threshold.

In our neural network, each neuron is composed of synapses, dendrites, and a single soma. Synapses take in spikes, dendrites process them, and the soma fires out new spikes. Our neurons are organized as "dendritic arbors". A dendritic arbor is a directed tree where the nodes are dendrites and the soma is the root. In graph theory, a graph is a collection of nodes and edges, and a tree is a graph with no cycles.

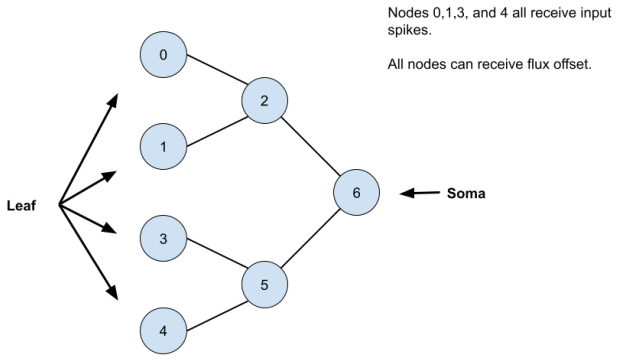

Figure 1: Diagram of basic neuron's dendritic arbor. There are four input dendrites and a single soma.

As you see in Figure 1, leaf dendrites get spikes from outside and then pass on flux and signal to dendrites further along the tree. Finally, once enough signal is integrated inside the soma, a spike is fired out after a signal threshold is passed. Dendrites have weighted edges that scale the flux that is passed on. Inside a neuron, information is carried by the signal and flux values. For inter-neuron communication, information is passed by spikes. The neurons can be organized into any kind of topology.

Also note that in our simulation each nueron is just a binary tree of six dendrites and one soma, but this could be changed.

In the neuromorphic computer, a neuron has synapses, dendrites, and a soma. Each part is a superconducting electronic circuit. So an entire neuron has many circuits together. Synapses receive spikes in the form of photons. First, a photon from another neuron hits a single photon detector (SPD) of a synapse. That synapse is connected to a dendrite that then starts integrating signal. Then flux gets passed on to later dendrites. Once enough signal is integrated inside the soma a photon is fired out to another neuron.

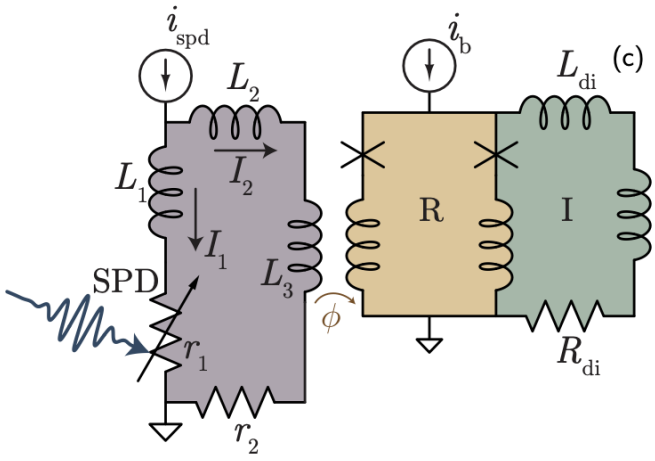

Figure 2: Circuit diagram of synapse with SPD, connected to dendrite. Adapted from [1]

Mathematically a dendrite is modeled as a first order ODE, which is a leaky integrator for current.

Each dendrite takes flux as input, as in Figure 2. We can abstract the circuit details in the simulation and just treat each dendrite and soma as functions that take in flux and output current. We will simulate the behavior of the circuits, and we just need to use the equations in the simulation implementation. The bottom line is that dendrites integrate signal, pass on flux, and eventually a spike is fired [1].

It might seem logical to create different dendrite and soma objects to simulate on, but while that is good for understanding what is occurring, it is not good for optimizing performance. Instead, we used large matrices and vectors to carry all of the flux and signal data, which is ideal for GPUs.

Learning on this network can be done in a number of ways. The most obvious is to change the weight strength of the edges but this is hard to do in the actual neuromorphic computer. Another way to learn is by adding flux offsets. Just how in an artificial neural network the basic equation is $wx + b$, instead of changing weights we can change the b term.

The simulation was written using Python and the Numba and CuPy libraries. CuPy is a combination of the CUDA and Numpy libraries for Python. The program was run on the NIST ENKI computing cluster.

2 Methods

The first thing to do is updating the flux in all dendrites and somas according to the incoming signal, external spikes, and flux offsets.

Equation 1 is the flux to dendrite i from all dendrites $j = 1...n$. It is the sum of the signals from dendrites $j = 1...n$ multiplied by a scalar weight that

represents the mutual inductance.

$$\phi_i = \sum_{j=1}^{n} J_{ij} s_j. \tag{1}$$

$$\phi_i = \sum_{j=1}^{n} (J_{ij} s_j) + s_{\text{ext}} + \phi_{\text{offset}}. \tag{2}$$

Equation 2 is similar but includes external spike signals and flux offset. The external signals come from other neurons firing spikes, and the flux offset is what we will use for learning. In the neuromorphic computer the flux offset is added to a dendrite circuit by another circuit.

Then we have to update the signal in a dendrite according to the previous flux and signal of the dendrite using Equation 3.

$$s_{\tau+1} = s_\tau \left(1 - \triangle\tau \frac{\alpha}{\beta}\right) + \frac{\triangle\tau}{\beta} r(\phi_\tau, s_\tau). \tag{3}$$

τ is the timestep, α, β are predetermined constants, and r is a function that calculates the flux and signal based on the properties of the superconducting components of the neuromorphic computer. In the original non-gpu simulation, r was a pre-calculated array that was indexed into. The approximation function is not the same shape but it is similar.

All of the previous equations would take a long time to compute if calculated sequentially. But since almost all of these are simple addition and multiplication operations GPUs are uniquely suited to computing all of these operations in parallel.

Then we just need to check all the somas for spiking behavior and update accordingly. This is a bottleneck in the simulation because we have to iterate through all the somas to check if they are ready to spike, and since the spike is a relatively more complex curve it does not take full advantage of the GPUs abilities. In the code we just reference an array that holds the values of the spike curve.

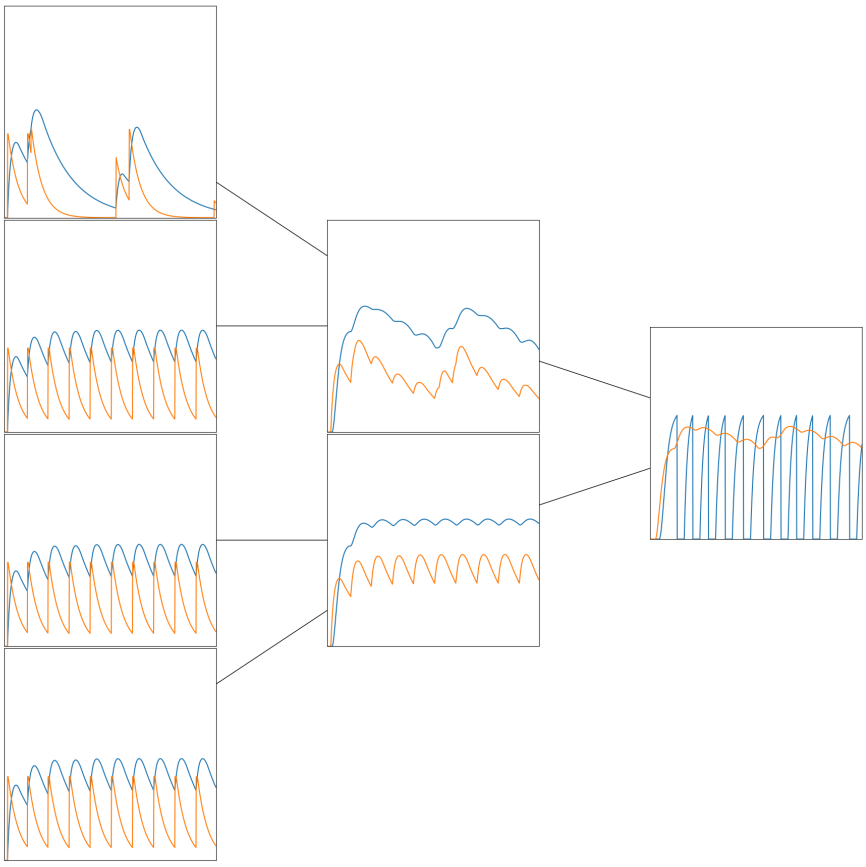

Figure 3: Tree diagram of dendrites and soma. Flux is orange, signal is blue.

2.1 GPU Implementation

We primarily used the CuPy library in Python which is a combination of the NumPy library with the CUDA programming language used in NVIDIA GPUs.

First, we initialize arrays in the GPU memory. It is time consuming to move data from CPU to GPU and vice versa, so minimizing the amount of swaps is important. To actually use the GPUs in the code we need to allocate one from Enki and then just run the cupy code.

The functions use array operations instead of for loops. Array operations can be processed in parallel in $O(n)$ time instead of nested loops which can take $O(n^2)$ Loops also have an associated overhead that we did not have to deal with in this case.

3 Results

We ran the simulation and measured the runtimes based on the number of simulation timesteps, dendrites, and neuron connectivity changed. We also accounted for the time difference in tracking the flux and signal, because tracking required more GPU space to be used.

Figure 4 represents simulations run on 1000 timesteps per simulation and a connectivity of 1 where all the neurons were chained together in a circle. And half of the neurons receive the same input at the same time, while the other half receive a weaker input at the same time.

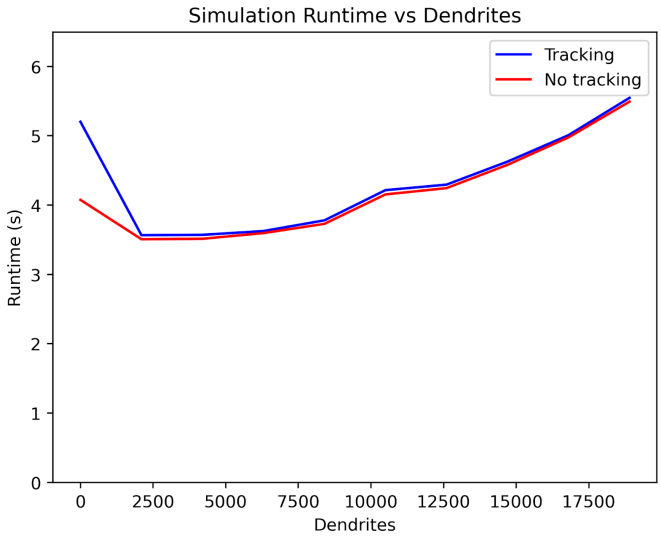

Figure 4: Graph of the simulation runtime as the number of dendrites increases.

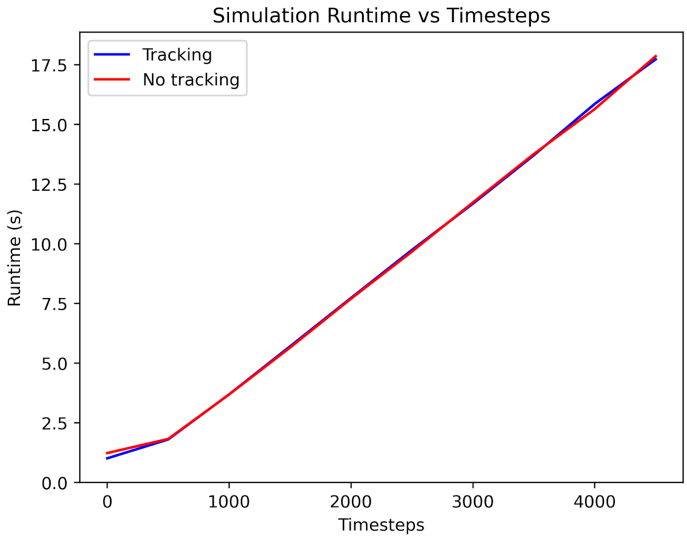

Figure 5: Graph of the simulation runtime as the number of timesteps increases. 1000 neurons, each neuron connected in a circle.

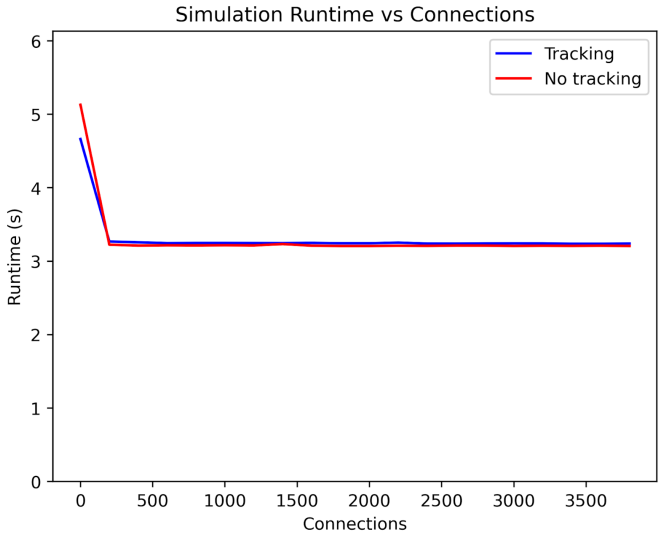

Figure 6: Graph of the simulation runtime as the number of neuron connections increases. 1000 neurons and 1000 timesteps.

The non-tracking version of the simulation that did not save every flux and signal value performed slightly faster in all three cases. The increase in runtime is linear for all three cases. The reason that the connections caused runtime to grow so much is due to how the simulation deals with spikes.

4 Discussion

Our GPU based simulation ran **x times faster** compared to a non GPU version.

We also used an approximation function for r to simulate the superconducting properties of the neuromorphic computer. We found that GPUs did speed up the simulation quite a bit.

The most time-consuming operation was the multiplication of the signal vector by the weight matrix for each time step. This calculation had a time complexity of $O(tn^2)$, where t was the number of timesteps and n was the number of dendrites. The Big-O notation is important but it seems for the amount of timesteps and dendrites tested here the coefficient was important for these smaller tests.

All the other operations are on the order of $O(n)$. The graphs also confirm this analysis. Figure 5 follows a linear trend which is because it has a time complexity of $O(t)$. Figure 6 is flat because it has a time complexity of $O(1)$, once all leaf nodes are receiving spikes, having more connected neurons will not increased the necessary computation because .

The space complexity is $O(n(n+t))$ because we utilize an n-by-n matrix to store all the weight values and a t-by-n matrix to store the fluxes and signals. Everything else is stored in n-length arrays so the n^2 term dominates.

Overall, using GPUs and writing the code to be based on arrays instead of classes allowed for significant speed improvements.

5 References

[1]J. M. Shainline, B. A. Primavera, and S. Khan, "Phenomenological model of superconducting optoelectronic loop neurons," Phys. Rev. Res., vol. 5, no. 1, p. 013164, Mar. 2023, doi: 10.1103/PhysRevResearch.5.013164.